《计算机图形学及人机交互》

作 业 报 告

学　　号	2023112621
姓　　名	程泽凯
年　　级	2023 级
专　　业	计算机科学与技术
院　　系	计算机与人工智能学院

2025~2026 第 1 学期

目录

1 Bresenham 圆绘制算法（40 分） 2
1.1 算法文字描述（15 分） . . . 2
1.2 计算示例（10 分） . . . 5
1.3 Rust 实现（15 分） . . . 11

2 多边形填充算法（60 分） 12
2.1 扫描线填充算法 . . . 12
2.1.1 算法描述（10 分） . . . 12
2.1.2 Rust 实现（15 分） . . . 14
2.1.3 复杂度分析（5 分） . . . 15
2.2 栅栏填充算法 . . . 17
2.2.1 算法描述（10 分） . . . 17
2.2.2 Rust 实现（15 分） . . . 19
2.2.3 复杂度分析（5 分） . . . 19
2.3 算法时间复杂度比较 . . . 22
2.3.1 多边形填充算法对比 . . . 22
2.3.2 与 Bresenham 圆算法的比较 . . . 24

3 运行结果 26
3.1 程序运行截图 . . . 26
3.2 运行结果分析 . . . 27
3.2.1 1. Bresenham 圆绘制算法验证 . . . 27
3.2.2 2. 多边形填充算法测试 . . . 28
3.2.3 3. 算法性能对比总结 . . . 29
3.2.4 4. 单元测试结果 . . . 29
3.2.5 5. 测试截图 . . . 30
3.2.6 6. 完整运行截图 . . . 32
3.3 结论 . . . 34

1 Bresenham 圆绘制算法（40 分）

1.1 算法文字描述（15 分）

算法思想：
Bresenham 圆绘制算法是一种增量式整数运算算法，通过利用圆的对称性和整数判别式来高效地绘制圆。该算法避免了浮点运算和复杂的平方根计算，仅使用加法和位移操作。

1. 圆的隐式方程
对于圆心在 (x_c, y_c)、半径为 r 的圆，其隐式方程为：

$$F(x,y) = (x - x_c)^2 + (y - y_c)^2 - r^2 = 0 \tag{1}$$

为简化分析，考虑圆心在原点的圆：$F(x,y) = x^2 + y^2 - r^2 = 0$

2. 八分对称性
圆具有八分对称性，即：

- 关于 x 轴对称：$(x,y) \leftrightarrow (x,-y)$
- 关于 y 轴对称：$(x,y) \leftrightarrow (-x,y)$
- 关于原点对称：$(x,y) \leftrightarrow (-x,-y)$
- 关于直线 $y = x$ 对称：$(x,y) \leftrightarrow (y,x)$

因此，只需计算第一个八分圆（从 $(0,r)$ 到 $(r/\sqrt{2}, r/\sqrt{2})$，即 $0° \sim 45°$）上的点，其余 7 个八分圆上的点可通过对称变换得到。在此区间内，满足 $x \leq y$。

3. 决策参数推导
在绘制过程中，假设当前点为 (x_k, y_k)，下一个点有两种选择：

- $P_1 = (x_k + 1, y_k)$：水平向右移动
- $P_2 = (x_k + 1, y_k - 1)$：右下移动

引入决策参数 d_k 来判断选择哪个点：

$$d_k = F(x_k + 1, y_k - 0.5) = (x_k + 1)^2 + (y_k - 0.5)^2 - r^2 \tag{2}$$

- 若 $d_k < 0$：说明中点在圆内，实际圆弧离 P_1 更近，选择 P_1，即 y 不变
- 若 $d_k \geq 0$：说明中点在圆外，实际圆弧离 P_2 更近，选择 P_2，即 y 减 1

4. 决策参数的增量计算

（**1**）初始值推导

当 $x_0 = 0, y_0 = r$ 时，需要计算第一个中点 $(1, r - 0.5)$ 的决策参数：

$$\begin{aligned} d_0 &= F(1, r - 0.5) && (3)\\ &= 1^2 + (r - 0.5)^2 - r^2 && (4)\\ &= 1 + r^2 - r + 0.25 - r^2 && (5)\\ &= 1.25 - r && (6) \end{aligned}$$

为避免浮点数运算，我们可以：

- 将决策参数乘以 2：$d_0' = 2d_0 = 2.5 - 2r \approx 2 - 2r$
- 或者使用近似：$d_0 = 1 - r$
- 实际使用：$d_0 = 3 - 2r$（整数，便于计算）

（**2**）增量更新推导

决策参数的更新采用增量方式，避免重复计算。

情况 **1**：若 $d_k < 0$（选择 $P_1 = (x_k + 1, y_k)$，水平右移）

下一个中点为 $(x_k + 2, y_k - 0.5)$，计算 d_{k+1}：

$$\begin{aligned} d_{k+1} &= F(x_k + 2, y_k - 0.5) && (7)\\ &= (x_k + 2)^2 + (y_k - 0.5)^2 - r^2 && (8)\\ &= \left[(x_k + 1)^2 + (y_k - 0.5)^2 - r^2\right] + 2(x_k + 1) + 1 && (9)\\ &= d_k + 2(x_k + 1) + 1 && (10)\\ &= d_k + 2x_k + 3 && (11) \end{aligned}$$

若使用 $2d$ 形式（决策参数乘以 2），则：

$$\boxed{d_{k+1} = d_k + 4x_k + 6} \quad (12)$$

情况 **2**：若 $d_k \geq 0$（选择 $P_2 = (x_k + 1, y_k - 1)$，右下移动）

下一个中点为 $(x_k+2, y_k-1.5)$，计算 d_{k+1}：

$$
\begin{aligned}
d_{k+1} &= F(x_k+2, y_k-1.5) && (13)\\
&= (x_k+2)^2 + (y_k-1.5)^2 - r^2 && (14)\\
&= \left[(x_k+1)^2 + (y_k-0.5)^2 - r^2\right] + 2(x_k+1) + 1 - 2y_k + 1 && (15)\\
&= d_k + 2(x_k+1) + 1 - 2y_k + 1 && (16)\\
&= d_k + 2(x_k - y_k + 1) + 2 && (17)\\
&= d_k + 2x_k - 2y_k + 4 && (18)
\end{aligned}
$$

若使用 $2d$ 形式，则：

$$\boxed{d_{k+1} = d_k + 4(x_k - y_k) + 10} \tag{19}$$

（**3**）增量计算总结

条件	选择的点	决策参数更新
$d_k < 0$	(x_k+1, y_k)	$d_{k+1} = d_k + 4x_k + 6$
$d_k \geq 0$	(x_k+1, y_k-1)	$d_{k+1} = d_k + 4(x_k - y_k) + 10$

5. 算法流程

完整的 **Bresenham** 圆绘制算法步骤如下：

1. 初始化：
   - $x = 0$，$y = r$
   - $d = 3 - 2r$
2. 循环绘制（当 $x \leq y$ 时）：
   - 根据当前点 (x, y) 绘制 8 个对称点：

$$
\begin{aligned}
&(x_c + x, y_c + y), \quad (x_c + y, y_c + x)\\
&(x_c - x, y_c + y), \quad (x_c - y, y_c + x)\\
&(x_c + x, y_c - y), \quad (x_c + y, y_c - x)\\
&(x_c - x, y_c - y), \quad (x_c - y, y_c - x)
\end{aligned}
$$

   - 判断并更新：
     - 若 $d < 0$：$d \leftarrow d + 4x + 6$
     - 若 $d \geq 0$：$d \leftarrow d + 4(x - y) + 10$，$y \leftarrow y - 1$

- $x \leftarrow x+1$

3. 终止条件：当 $x > y$ 时停止（已完成第一个八分圆）

6. 算法特点与复杂度分析

- 优点：
  - 仅使用整数加法和位移运算，速度快
  - 利用对称性，只需计算 1/8 圆周，生成 8 倍的点
  - 避免了浮点运算和三角函数计算
  - 误差控制在 1 个像素以内
- 缺点：
  - 仅适用于标准圆，不能直接用于椭圆
  - 需要特殊处理对称点的重复问题
- 复杂度分析：
  - 时间复杂度：$O(r)$，其中 r 是圆的半径。只需遍历第一个八分圆的 $r/\sqrt{2}$ 个点。
  - 空间复杂度：$O(1)$，仅需要常数个变量存储当前状态。
- 应用场景：
  - 计算机图形学中的圆形绘制
  - 游戏开发中的碰撞检测
  - CAD 系统中的圆形对象渲染

1.2 计算示例（10 分）

问题：圆心 (2,2)，半径 10，求 x=8 时的 y 值
理论验证：$(8-2)^2 + (y-2)^2 = 100 \Rightarrow y = 10$ 或 $y = -6$
Bresenham 计算过程：

初始化：

$$x = 0$$
$$y = r = 10$$
$$d = 3 - 2r = 3 - 2 \times 10 = -17$$

详细迭代过程：

下面逐步展示每次迭代的完整计算过程，包括决策判断、参数更新、几何意义和验证。

1. 第 **1** 次迭代（$x = 0, y = 10$）：

   - 当前点：$(x, y) = (0, 10)$，相对圆心的坐标
   - 绝对坐标：$(x_c + x, y_c + y) = (2, 12)$
   - 决策参数：$d = -17$
   - 中点坐标：$(x + 1, y - 0.5) = (1, 9.5)$
   - 中点判断：$d = F(1, 9.5) = 1^2 + 9.5^2 - 10^2 = 1 + 90.25 - 100 = -8.75 < 0$
     说明中点在圆内，选择 $P_1 = (1, 10)$（水平右移）
   - 决策：$d < 0$，使用增量公式 $d_{new} = d + 4x + 6$
   - 参数更新：

     $$d_{new} = -17 + 4 \times 0 + 6 = -17 + 0 + 6 = -11$$
     $$x_{new} = 0 + 1 = 1$$
     $$y_{new} = 10\text{（不变）}$$

   - 点验证：$x^2 + y^2 = 0^2 + 10^2 = 100 = r^2$ ✓（精确在圆上）
   - 继续条件：$x = 0 \leq y = 10$ ✓

2. 第 **2** 次迭代（$x = 1, y = 10$）：

   - 当前点：$(1, 10)$
   - 绝对坐标：$(3, 12)$
   - 决策参数：$d = -11$
   - 中点坐标：$(2, 9.5)$
   - 中点判断：$F(2, 9.5) = 4 + 90.25 - 100 = -5.75 < 0$
     中点在圆内，选择 $(2, 10)$

- 决策：$d = -11 < 0$
- 参数更新：

$$d_{new} = -11 + 4 \times 1 + 6 = -11 + 4 + 6 = -1$$
$$x_{new} = 1 + 1 = 2$$
$$y_{new} = 10$$

- 点验证：$1^2 + 10^2 = 101 \approx 100$（误差 1，接近圆）
- 继续条件：$x = 1 \leq y = 10$ ✓

3. 第 **3** 次迭代（$x = 2, y = 10$）：

- 当前点：$(2, 10)$
- 绝对坐标：$(4, 12)$
- 决策参数：$d = -1$
- 中点坐标：$(3, 9.5)$
- 中点判断：$F(3, 9.5) = 9 + 90.25 - 100 = -0.75 < 0$
  中点在圆内（接近边界），选择 $(3, 10)$
- 决策：$d = -1 < 0$
- 参数更新：

$$d_{new} = -1 + 4 \times 2 + 6 = -1 + 8 + 6 = 13$$
$$x_{new} = 2 + 1 = 3$$
$$y_{new} = 10$$

- 点验证：$2^2 + 10^2 = 104 \approx 100$（误差 4）
- 继续条件：$x = 2 \leq y = 10$ ✓

4. 第 **4** 次迭代（$x = 3, y = 10$）：

- 当前点：$(3, 10)$
- 绝对坐标：$(5, 12)$
- 决策参数：$d = 13$
- 中点坐标：$(4, 9.5)$
- 中点判断：$F(4, 9.5) = 16 + 90.25 - 100 = 6.25 > 0$
  中点在圆外，选择 $P_2 = (4, 9)$（右下移动）

- 决策：$d = 13 \geq 0$，使用增量公式 $d_{new} = d + 4(x - y) + 10$
- 参数更新：

$$\begin{aligned} d_{new} &= 13 + 4 \times (3 - 10) + 10 \\ &= 13 + 4 \times (-7) + 10 \\ &= 13 - 28 + 10 = -5 \\ y_{new} &= 10 - 1 = 9 \\ x_{new} &= 3 + 1 = 4 \end{aligned}$$

- 点验证：$3^2 + 10^2 = 109 \neq 100$（误差较大，但下一个点会更接近）
- 继续条件：$x = 3 \leq y = 10$ ✓

5. 第 **5** 次迭代（$x = 4, y = 9$）：

- 当前点：$(4, 9)$
- 绝对坐标：$(6, 11)$
- 决策参数：$d = -5$
- 中点坐标：$(5, 8.5)$
- 中点判断：$F(5, 8.5) = 25 + 72.25 - 100 = -2.75 < 0$
  中点在圆内，选择 $(5, 9)$
- 决策：$d = -5 < 0$
- 参数更新：

$$\begin{aligned} d_{new} &= -5 + 4 \times 4 + 6 = -5 + 16 + 6 = 17 \\ x_{new} &= 4 + 1 = 5 \\ y_{new} &= 9 \end{aligned}$$

- 点验证：$4^2 + 9^2 = 16 + 81 = 97 \approx 100$（误差 3）
- 继续条件：$x = 4 \leq y = 9$ ✓

6. 第 **6** 次迭代（$x = 5, y = 9$）：

- 当前点：$(5, 9)$
- 绝对坐标：$(7, 11)$
- 决策参数：$d = 17$
- 中点坐标：$(6, 8.5)$

- 中点判断：$F(6, 8.5) = 36 + 72.25 - 100 = 8.25 > 0$
  中点在圆外，选择 $(6, 8)$
- 决策：$d = 17 \geq 0$
- 参数更新：

$$\begin{aligned} d_{new} &= 17 + 4 \times (5 - 9) + 10 \\ &= 17 + 4 \times (-4) + 10 \\ &= 17 - 16 + 10 = 11 \\ y_{new} &= 9 - 1 = 8 \\ x_{new} &= 5 + 1 = 6 \end{aligned}$$

- 点验证：$5^2 + 9^2 = 25 + 81 = 106 \neq 100$（误差 6）
- 继续条件：$x = 5 \leq y = 9$ ✓

7. 第 **7** 次迭代（$x = 6, y = 8$）：

- 当前点：$(6, 8)$
- 绝对坐标：$(8, 10)$ ← 这是我们要找的点！
- 决策参数：$d = 11$
- 中点坐标：$(7, 7.5)$
- 中点判断：$F(7, 7.5) = 49 + 56.25 - 100 = 5.25 > 0$
  中点在圆外，选择 $(7, 7)$
- 决策：$d = 11 \geq 0$
- 参数更新：

$$\begin{aligned} d_{new} &= 11 + 4 \times (6 - 8) + 10 \\ &= 11 + 4 \times (-2) + 10 \\ &= 11 - 8 + 10 = 13 \\ y_{new} &= 8 - 1 = 7 \\ x_{new} &= 6 + 1 = 7 \end{aligned}$$

- 点验证：$6^2 + 8^2 = 36 + 64 = 100 = r^2$ ✓（精确在圆上！）
- 继续条件：$x = 6 \leq y = 8$ ✓

8. 第 **8** 次迭代（$x = 7, y = 7$）：

- 当前点：$(7,7)$
- 绝对坐标：$(9,9)$
- 决策参数：$d = 13$
- 点验证：$7^2 + 7^2 = 49 + 49 = 98 \approx 100$（误差 2）
- 终止判断：$x = 7 \leq y = 7$（边界情况），可以继续一次或终止
- 如果继续：$x_{new} = 8 > y = 7$ 或 $y_{new} = 6$，满足 $x > y$，终止

迭代过程汇总表：

迭代	x	y	d	决策	d_{new}	绝对坐标	$x^2 + y^2$
初始	0	10	-17	-	-	(2,12)	100
1	0	10	-17	$d < 0$	-11	(2,12)	100
2	1	10	-11	$d < 0$	-1	(3,12)	101
3	2	10	-1	$d < 0$	13	(4,12)	104
4	3	10	13	$d \geq 0$	-5	(5,12)	109
5	4	9	-5	$d < 0$	17	(6,11)	97
6	5	9	17	$d \geq 0$	11	(7,11)	106
7	6	8	11	$d \geq 0$	13	(8,10)	100
8	7	7	13	终止	-	(9,9)	98

表 1: Bresenham 圆算法迭代过程汇总（圆心 (2,2)，半径 10）

关键观察：

- 第 1 次和第 7 次迭代得到的点 $(0,10)$ 和 $(6,8)$ 精确在圆上（$x^2 + y^2 = 100$）
- 其他点都是近似点，但误差很小（最大误差 9，相对误差 9%）
- 决策参数 d 在正负之间交替变化，反映了实际圆弧在候选点之间的位置
- 当 d 从负变正时（第 4、6 次迭代），算法选择向下移动，使点更接近圆弧

利用八分对称性求解：
根据圆的八分对称性，对于第一象限内的点 (x_0, y_0)，可以得到 8 个对称点：

$$
\begin{aligned}
&(x_c + x_0, y_c + y_0), \quad (x_c + y_0, y_c + x_0) \\
&(x_c - x_0, y_c + y_0), \quad (x_c - y_0, y_c + x_0) \\
&(x_c + x_0, y_c - y_0), \quad (x_c + y_0, y_c - x_0) \\
&(x_c - x_0, y_c - y_0), \quad (x_c - y_0, y_c - x_0)
\end{aligned}
$$

当 $x_0 = 6, y_0 = 8$ 时（圆心为 $(2, 2)$）：

$$
\begin{aligned}
(2+6, 2+8) &= (8, 10) \quad \checkmark \\
(2+8, 2+6) &= (10, 8) \\
(2-6, 2+8) &= (-4, 10) \\
(2-8, 2+6) &= (-6, 8) \\
(2+6, 2-8) &= (8, -6) \quad \checkmark \\
(2+8, 2-6) &= (10, -4) \\
(2-6, 2-8) &= (-4, -6) \\
(2-8, 2-6) &= (-6, -4)
\end{aligned}
$$

因此，当 $x = 8$ 时，得到 $y = 10$ 或 $y = -6$。
结论：$\boxed{y = 10 \text{ 或} y = -6}$

1.3 Rust 实现（15 分）

```rust
pub fn bresenham_circle(cx: i32, cy: i32, r: i32) -> Vec<Point> {
    let mut points = Vec::new();
    let (mut x, mut y) = (0, r);
    let mut d = 3 - 2 * r;

    while x <= y {
        // 8个对称点
        points.push(Point::new(cx+x, cy+y));
        points.push(Point::new(cx+y, cy+x));
        points.push(Point::new(cx-x, cy+y));
        points.push(Point::new(cx-y, cy+x));
        points.push(Point::new(cx-x, cy-y));
        points.push(Point::new(cx-y, cy-x));
        points.push(Point::new(cx+x, cy-y));
        points.push(Point::new(cx+y, cy-x));

        if d < 0 {
            d += 4 * x + 6;
        } else {
            d += 4 * (x - y) + 10;
            y -= 1;
        }
```

```
        x += 1;
    }
    points
}
```

2 多边形填充算法（60 分）

2.1 扫描线填充算法

2.1.1 算法描述（10 分）

算法思想：
扫描线填充算法是一种基于扫描线的区域填充算法，通过水平扫描线从下到上逐行扫描多边形，计算扫描线与多边形边的交点，然后在交点间进行配对填充。该算法高效且适用于任意复杂的多边形。

1. 核心数据结构

（**1**）边表（**Edge Table, ET**）

边表是一个按扫描线 y 坐标组织的数组，$ET[y]$ 存储 $y_{min} = y$ 的所有非水平边。每条边记录以下信息：

- y_{max}：边的最大 y 坐标
- x：边与当前扫描线的交点的 x 坐标
- Δx：扫描线每增加 1 时，交点 x 坐标的增量（即斜率的倒数 $1/k$）

边的结构体表示为：

```
struct Edge {
    y_max: i32,       // 边的上端点y坐标
    x: f64,           // 当前扫描线交点的x坐标
    dx: f64,          // 斜率的倒数 1/k
}
```

（**2**）活性边表（**Active Edge Table, AET**）

活性边表存储与当前扫描线相交的所有边，按照交点的 x 坐标递增排序。AET 中的边满足：

$$y_{min} \leq y_{current} < y_{max} \tag{20}$$

2. 算法详细步骤

步骤 **1**：构造边表 **ET**

遍历多边形的所有顶点，对于每条边 (v_i, v_{i+1})：

1. 跳过水平边（$y_i = y_{i+1}$）
2. 确定边的下端点 y_{min} 和上端点 y_{max}
3. 计算斜率倒数：$\Delta x = \frac{x_{i+1}-x_i}{y_{i+1}-y_i}$
4. 将边信息插入 $ET[y_{min}]$ 中

特殊处理：对于顶点，若该顶点是多边形的局部极值点（如局部最大值或最小值），则只计入一条边；若是普通顶点，需要正确处理以避免重复计数。
步骤 **2**：初始化扫描

- 找到多边形的最小 y 坐标：$y = y_{min}$
- 初始化活性边表 $AET = \emptyset$

步骤 **3**：扫描线循环（从 y_{min} 到 y_{max}）
对于每条扫描线 y，执行以下操作：
（**1**）更新 **AET**

- 将 $ET[y]$ 中的所有边加入 AET
- 从 AET 中删除满足 $y \geq y_{max}$ 的边（这些边已经处理完毕）
- 按 x 坐标对 AET 进行排序

（**2**）填充像素
采用奇偶规则（Odd-Even Rule）或非零环绕数规则（Non-Zero Winding Rule）进行填充：

- 奇偶规则：从左到右扫描 AET，交点配对填充。第 1 和第 2 个交点之间填充，第 3 和第 4 个交点之间填充，依此类推。
- 具体实现：按顺序取 AET 中相邻的交点对 (x_i, x_{i+1})（i 为奇数），填充区间 $[\lceil x_i \rceil, \lfloor x_{i+1} \rfloor]$ 内的所有像素。

（**3**）更新交点 **x** 坐标
对于 AET 中的每条边，更新其与下一条扫描线的交点：

$$x_{new} = x_{old} + \Delta x \tag{21}$$

步骤 **4**：终止条件
当 $y > y_{max}$ 且 AET 为空时，算法结束。
3. 边界情况处理

- 水平边：不加入边表，因为水平边对填充没有贡献
- 共享顶点：对于共享顶点，使用” 下闭上开” 原则，即边的下端点包含在边中，上端点不包含，避免顶点被重复处理
- 退化多边形：处理面积为 0 的多边形（如共线点）

4. 算法特点

- 优点：
  - 利用扫描线的连贯性，避免重复计算
  - 仅需要维护活性边，空间效率高
  - 适用于复杂多边形和带孔多边形
- 缺点：
  - 需要构造和维护边表，实现相对复杂
  - 需要对 AET 进行排序，有一定的开销
- 时间复杂度：$O(n + h \cdot (e \log e + w))$
  - n：多边形顶点数
  - h：多边形高度（扫描线数量）
  - e：平均每条扫描线相交的边数
  - w：每条扫描线填充的像素宽度
- 空间复杂度：$O(n + e)$

2.1.2 Rust 实现（15 分）

```rust
pub fn scanline_fill(vertices: &[Point]) -> Vec<Point> {
    let mut points = Vec::new();
    let y_min = vertices.iter().map(|p| p.y).min().unwrap();
    let y_max = vertices.iter().map(|p| p.y).max().unwrap();

    // 构建边表...
    // 扫描线填充...

    for y in y_min..=y_max {
```

```
        // 加入新边，排序，配对填充，更新AET
    }
    points
}
```

2.1.3 复杂度分析（5 分）

时间复杂度详细分析：

我们逐步分析算法各部分的时间开销：

1. 构造边表 **ET**

```
// 遍历所有顶点，构造边表
for i in 0..n {                                    // O(n)
    let edge = (vertices[i], vertices[(i+1) % n]);
    if edge.y_min != edge.y_max {                  // 跳过水平边
        ET[edge.y_min].push(edge);                 // O(1) 插入
    }
}
```

时间：$O(n)$，其中 n 为顶点数（即边数）

2. 扫描线循环

```
for y in y_min..=y_max {                           // O(h) 循环h次
    // (a) 更新AET: 加入新边
    AET.extend(ET[y]);                             // O(e_y) 加入e_y条新边

    // (b) 删除失效的边
    AET.retain(|edge| edge.y_max > y);             // O(e) 遍历AET

    // (c) 对AET按x坐标排序
    AET.sort_by(|a, b| a.x.cmp(&b.x));             // O(e log e)

    // (d) 配对填充
    for i in (0..AET.len()).step_by(2) {           // O(w) 填充w个像素
        fill_line(AET[i].x, AET[i+1].x, y);
    }

    // (e) 更新交点x坐标
    for edge in &mut AET {                         // O(e)
        edge.x += edge.dx;
    }
```

```
}
```

每条扫描线的时间开销：

- 加入新边：$O(e_y)$，平均为 $O(e/h)$
- 删除失效边：$O(e)$
- 排序 AET：$O(e \log e)$
- 配对填充：$O(w)$，w 为填充宽度
- 更新交点：$O(e)$

单条扫描线总时间：$O(e \log e + w)$

3. 总时间复杂度推导

$$T_{total} = T_{ET} + T \tag{22}$$

$$= O(n) + \sum_{y=y_{min}}^{y_{max}} O(e \log e + w) \tag{23}$$

$$= O(n) + h \cdot O(e \log e + w) \tag{24}$$

$$= O(n + h(e \log e + w)) \tag{25}$$

其中：

- n：多边形边数
- $h = y_{max} - y_{min} + 1$：扫描线数量（多边形高度）
- e：平均每条扫描线的活性边数，通常 $e \ll n$
- w：平均每条扫描线的填充宽度

最坏情况：对于复杂多边形，$e \approx n$，时间复杂度为 $O(n + h(n \log n + w))$

最好情况：对于简单凸多边形，$e = 2$（每条扫描线只与 2 条边相交），时间复杂度为 $O(n + h \cdot w)$

空间复杂度分析：

- 边表 ET：存储所有非水平边，$O(n)$
- 活性边表 AET：存储当前扫描线的活性边，最坏情况 $O(e)$，其中 $e \leq n$
- 其他辅助变量：$O(1)$

总空间复杂度：$S = O(n + e) = O(n)$

2.2 栅栏填充算法

2.2.1 算法描述（10 分）

算法思想：
栅栏填充算法（Fence Fill）是扫描线填充算法的简化版本，其核心思想是对每条扫描线，直接遍历多边形的所有边，计算与该扫描线的交点，然后对交点排序并配对填充。该算法不维护边表和活性边表，实现更加简单直观。
1. 算法详细步骤
步骤 **1**：确定扫描范围
遍历多边形的所有顶点，找到最小和最大的 y 坐标：

$$y_{min} = \min\{y_i \mid i = 1, 2, \ldots, n\} \tag{26}$$

$$y_{max} = \max\{y_i \mid i = 1, 2, \ldots, n\} \tag{27}$$

步骤 **2**：扫描线循环
对于每条扫描线 $y \in [y_{min}, y_{max}]$：
（**1**）计算交点
遍历多边形的所有边 (v_i, v_{i+1})，判断边是否与扫描线相交：

- 跳过水平边：$y_i = y_{i+1}$
- 判断相交条件：$\min(y_i, y_{i+1}) \le y < \max(y_i, y_{i+1})$（使用” 下闭上开” 原则）
- 若相交，计算交点的 x 坐标：

$$x = x_i + \frac{(y - y_i) \cdot (x_{i+1} - x_i)}{y_{i+1} - y_i} \tag{28}$$

- 将 x 加入交点列表

（**2**）交点排序
将交点列表按 x 坐标递增排序：

$$\text{sort}(\{x_1, x_2, \ldots, x_k\}) \to \{x'_1, x'_2, \ldots, x'_k\} \quad \text{其中} \quad x'_1 \le x'_2 \le \cdots \le x'_k \tag{29}$$

（**3**）配对填充
按照奇偶规则进行填充：

- 将交点两两配对：$(x'_1, x'_2), (x'_3, x'_4), \ldots, (x'_{k-1}, x'_k)$（假设 k 为偶数）
- 对每一对 (x'_i, x'_{i+1})（i 为奇数），填充区间 $[\lceil x'_i \rceil, \lfloor x'_{i+1} \rfloor]$ 内的所有像素

步骤 **3**：终止
当所有扫描线处理完毕后，算法结束。

2. 与扫描线填充算法的比较

特性	扫描线填充	栅栏填充
数据结构	需要 ET 和 AET	仅需交点数组
实现复杂度	较复杂	简单
每条扫描线的工作	仅处理活性边	遍历所有边
时间复杂度	$O(n + h \cdot (e \log e + w))$	$O(h \cdot (n + k \log k + w))$
空间复杂度	$O(n + e)$	$O(k)$
适用场景	复杂多边形	简单多边形

其中：

- n：多边形边数
- h：扫描线数量
- e：平均每条扫描线的活性边数
- k：平均每条扫描线的交点数
- w：每条扫描线填充的像素宽度

3. 算法特点

- 优点：
  - 实现非常简单，无需维护复杂的数据结构
  - 代码短小，易于理解和调试
  - 空间开销小，仅需要存储当前扫描线的交点
- 缺点：
  - 每条扫描线都要遍历所有边，存在大量重复计算
  - 对于边数较多的复杂多边形，效率较低
  - 没有利用相邻扫描线的连贯性
- 适用场景：
  - 边数较少的简单多边形

- 对性能要求不高的场景
- 原型开发和教学演示

4. 优化建议

- 预先剔除水平边，减少判断次数
- 使用快速排序或基数排序对交点排序
- 对于静态多边形，可预计算边的包围盒，快速剔除不相交的边

2.2.2 Rust 实现（15 分）

```rust
pub fn fence_fill(vertices: &[Point]) -> Vec<Point> {
    for y in y_min..=y_max {
        for each edge {
            if edge intersects y {
                compute x_intersect
            }
        }
        sort and fill
    }
}
```

2.2.3 复杂度分析（5 分）

时间复杂度详细分析：

我们逐步分析栅栏填充算法各部分的时间开销：

1. 确定扫描范围

```rust
let y_min = vertices.iter().map(|p| p.y).min().unwrap();  // O(n)
let y_max = vertices.iter().map(|p| p.y).max().unwrap();  // O(n)
```

时间：$O(n)$，遍历所有顶点找最小和最大 y 坐标

2. 扫描线循环

```rust
for y in y_min..=y_max {                        // O(h) 循环h次
    let mut intersections = Vec::new();

    // (a) 计算所有交点
    for i in 0..n {                             // O(n) 遍历所有边
```

```
        let (p1, p2) = (vertices[i], vertices[(i+1) % n]);

        // 判断边是否与扫描线相交
        if p1.y == p2.y {                    // O(1) 跳过水平边
            continue;
        }

        if min(p1.y, p2.y) <= y && y < max(p1.y, p2.y) {
            // 计算交点x坐标
            let x = p1.x + (y - p1.y) * (p2.x - p1.x)
                        / (p2.y - p1.y);     // O(1) 算术运算
            intersections.push(x);           // O(1) 插入
        }
    }
    // 计算交点总时间: O(n)

    // (b) 对交点排序
    intersections.sort_by(|a, b| a.partial_cmp(b));  // O(k log k)
    // k为交点数量, 通常k << n

    // (c) 配对填充
    for i in (0..intersections.len()).step_by(2) {   // O(w)
        let x_start = intersections[i].ceil() as i32;
        let x_end = intersections[i+1].floor() as i32;
        for x in x_start..=x_end {                   // O(w_i)
            points.push(Point::new(x, y));
        }
    }
    // 填充总时间: O(w), w为该扫描线的填充宽度
}
```

每条扫描线的时间开销：

- 遍历所有边计算交点：$O(n)$
- 对 k 个交点排序：$O(k \log k)$，其中 k 是交点数量
- 配对填充：$O(w)$，w 是填充宽度

单条扫描线总时间：$O(n + k \log k + w)$

3. 总时间复杂度推导

$$T_{total} = T \quad + T \tag{30}$$

$$= O(n) + \sum_{y=y_{min}}^{y_{max}} O(n + k\log k + w) \tag{31}$$

$$= O(n) + h \cdot O(n + k\log k + w) \tag{32}$$

$$= O(n + h(n + k\log k + w)) \tag{33}$$

$$= O(h(n + k\log k + w)) \tag{34}$$

因为通常 $h \gg 1$，初始化的 $O(n)$ 被主循环吸收。

其中：

- n：多边形边数
- $h = y_{max} - y_{min} + 1$：扫描线数量
- k：平均每条扫描线的交点数，对于简单多边形 $k \approx 2$，对于复杂多边形 k 可能较大
- w：平均每条扫描线的填充宽度

最坏情况：对于复杂多边形，$k \approx n$（所有边都与某条扫描线相交），时间复杂度为 $O(h \cdot n\log n)$

最好情况：对于简单凸多边形，$k = 2$（每条扫描线恰好与 2 条边相交），时间复杂度为 $O(h(n + w))$

与扫描线填充的对比：

- 扫描线填充：$O(n + h(e\log e + w))$，其中 e 是活性边数
- 栅栏填充：$O(h(n + k\log k + w))$
- 栅栏填充每次都要遍历 n 条边，而扫描线只处理活性边 $e \ll n$
- 当 $h \cdot n \gg n + h \cdot e$ 时，扫描线填充更高效

空间复杂度分析：

- 交点数组：存储当前扫描线的交点，最坏情况 $O(k)$，其中 $k \leq n$
- 输出点集：$O(h \cdot w)$（如果需要返回所有填充点）
- 其他辅助变量：$O(1)$

总空间复杂度：$S = O(k) = O(n)$（不计输出）
实际性能分析：
对于一个 $n = 100$ 边、$h = 200$ 高度的多边形：

- 扫描线填充（假设 $e = 4$）：$100 + 200 \times (4 \log 4 + w) \approx 100 + 1600 + 200w$
- 栅栏填充（假设 $k = 4$）：$200 \times (100 + 4 \log 4 + w) \approx 20000 + 1600 + 200w$
- 扫描线填充快约 10 倍！

2.3 算法时间复杂度比较

2.3.1 多边形填充算法对比

1. 时间复杂度对比表

算法	时间复杂度	最好情况	最坏情况
扫描线填充	$O(n + h(e \log e + w))$	$O(n + h \cdot w)$	$O(n + h(n \log n + w))$
栅栏填充	$O(h(n + k \log k + w))$	$O(h(n + w))$	$O(h \cdot n \log n)$

表 2: 多边形填充算法时间复杂度对比

参数说明：

- n：多边形边数
- h：多边形高度（扫描线数量）
- e：平均每条扫描线的活性边数（扫描线算法）
- k：平均每条扫描线的交点数（栅栏算法）
- w：平均每条扫描线的填充宽度

2. 渐近性能分析
我们分析三个主要因素对算法性能的影响：
（**1**）边数 n 的影响

- 扫描线填充：$T = O(n) + O(h \cdot e \log e + h \cdot w)$

  构造边表需要 $O(n)$ 时间，但在扫描阶段，每条扫描线只处理 e 条活性边（$e \ll n$），因此对 n 不敏感。

- 栅栏填充：$T = O(h \cdot n + h \cdot k \log k + h \cdot w)$

  每条扫描线都要遍历所有 n 条边，因此时间与 n 成正比，对边数非常敏感。

性能比值：

$$\frac{T}{T} \approx \frac{h \cdot n}{n + h \cdot e} \approx \frac{h \cdot n}{h \cdot e} = \frac{n}{e} \tag{35}$$

当 $n \gg e$ 时，扫描线填充显著快于栅栏填充。

（**2**）多边形高度 h 的影响

两种算法的时间复杂度都与 h 线性相关，但系数不同：

$$T \approx O(n) + h \cdot C_1 \quad \text{其中} C_1 = O(e \log e + w) \tag{36}$$

$$T \approx h \cdot C_2 \quad \text{其中} C_2 = O(n + k \log k + w) \tag{37}$$

由于 $C_2 \gg C_1$（因为 $n \gg e$），栅栏填充的性能随 h 增加而快速下降。

（**3**）多边形复杂度的影响

多边形类型	活性边数 e	扫描线复杂度	栅栏复杂度
凸多边形	$e = 2$	$O(n + h(2 + w))$	$O(h(n + 2 + w))$
简单多边形	$e \approx 2 \sim 4$	$O(n + h(4 + w))$	$O(h(n + 4 + w))$
星形多边形	$e \approx n/2$	$O(n + h(n/2 + w))$	$O(h(n + n/2 + w))$
复杂多边形	$e \approx n$	$O(n(1 + h))$	$O(h \cdot n)$

表 3: 不同多边形类型的复杂度对比

3. 具体场景性能估算

假设填充宽度 $w \approx 100$ 像素，我们比较几种典型场景：

场景 **1**：简单三角形（$n = 3, h = 100, e = 2, k = 2$）

$$T = 3 + 100 \times (2 \log 2 + 100) \approx 10203 \tag{38}$$

$$T = 100 \times (3 + 2 \log 2 + 100) \approx 10500 \tag{39}$$

$$\text{性能比} \approx 1.03 \quad \text{（接近）} \tag{40}$$

场景 **2**：简单矩形（$n = 4, h = 200, e = 2, k = 2$）

$$T = 4 + 200 \times (2 \log 2 + 100) \approx 20404 \tag{41}$$

$$T = 200 \times (4 + 2 \log 2 + 100) \approx 21200 \tag{42}$$

$$\text{性能比} \approx 1.04 \quad \text{（接近）} \tag{43}$$

场景 **3**：中等复杂多边形（$n = 50, h = 200, e = 6, k = 6$）

$$T = 50 + 200 \times (6 \log 6 + 100) \approx 23500 \tag{44}$$

$$T = 200 \times (50 + 6 \log 6 + 100) \approx 33500 \tag{45}$$

$$\text{性能比} \approx 1.43 \quad \text{（栅栏慢 43\%）} \tag{46}$$

场景 **4**：复杂多边形（$n = 200, h = 300, e = 10, k = 10$）

$$T = 200 + 300 \times (10 \log 10 + 100) \approx 36900 \tag{47}$$

$$T = 300 \times (200 + 10 \log 10 + 100) \approx 96900 \tag{48}$$

$$\text{性能比} \approx 2.63 \text{ （栅栏慢 } 163\%\text{）} \tag{49}$$

场景 **5**：高复杂度多边形（$n = 1000, h = 500, e = 20, k = 20$）

$$T = 1000 + 500 \times (20 \log 20 + 100) \approx 64500 \tag{50}$$

$$T = 500 \times (1000 + 20 \log 20 + 100) \approx 563000 \tag{51}$$

$$\text{性能比} \approx 8.73 \text{ （栅栏慢 } 773\%\text{）} \tag{52}$$

4. 算法选择建议

场景	推荐算法	理由
简单多边形（$n \leq 10$）	栅栏填充	实现简单,性能差异小
中等复杂度（$10 < n \leq 50$）	扫描线填充	性能开始显著优于栅栏
复杂多边形（$n > 50$）	扫描线填充	性能优势明显
原型开发/教学	栅栏填充	代码简洁易懂
生产环境	扫描线填充	通用性强，性能稳定
实时图形应用	扫描线填充	需要高性能

表 4: 算法选择建议

5. 总结

- 扫描线填充是工业标准算法，适用于各种复杂度的多边形，性能随边数增长缓慢
- 栅栏填充适合简单多边形和教学场景，实现简单但性能随边数线性下降
- 当 $n > 20$ 且 $h > 100$ 时，扫描线填充的优势非常明显
- 对于 $n \geq 100$ 的复杂多边形，扫描线填充可以快 5-10 倍

2.3.2 与 Bresenham 圆算法的比较

虽然 Bresenham 圆算法和多边形填充算法解决不同的问题，但可以从复杂度角度进行对比：

算法	时间复杂度	空间复杂度	主要操作
Bresenham 圆	$O(r)$	$O(1)$	整数加法
扫描线填充	$O(n + h(e \log e + w))$	$O(n + e)$	排序 + 填充
栅栏填充	$O(h(n + k \log k + w))$	$O(k)$	遍历 + 排序

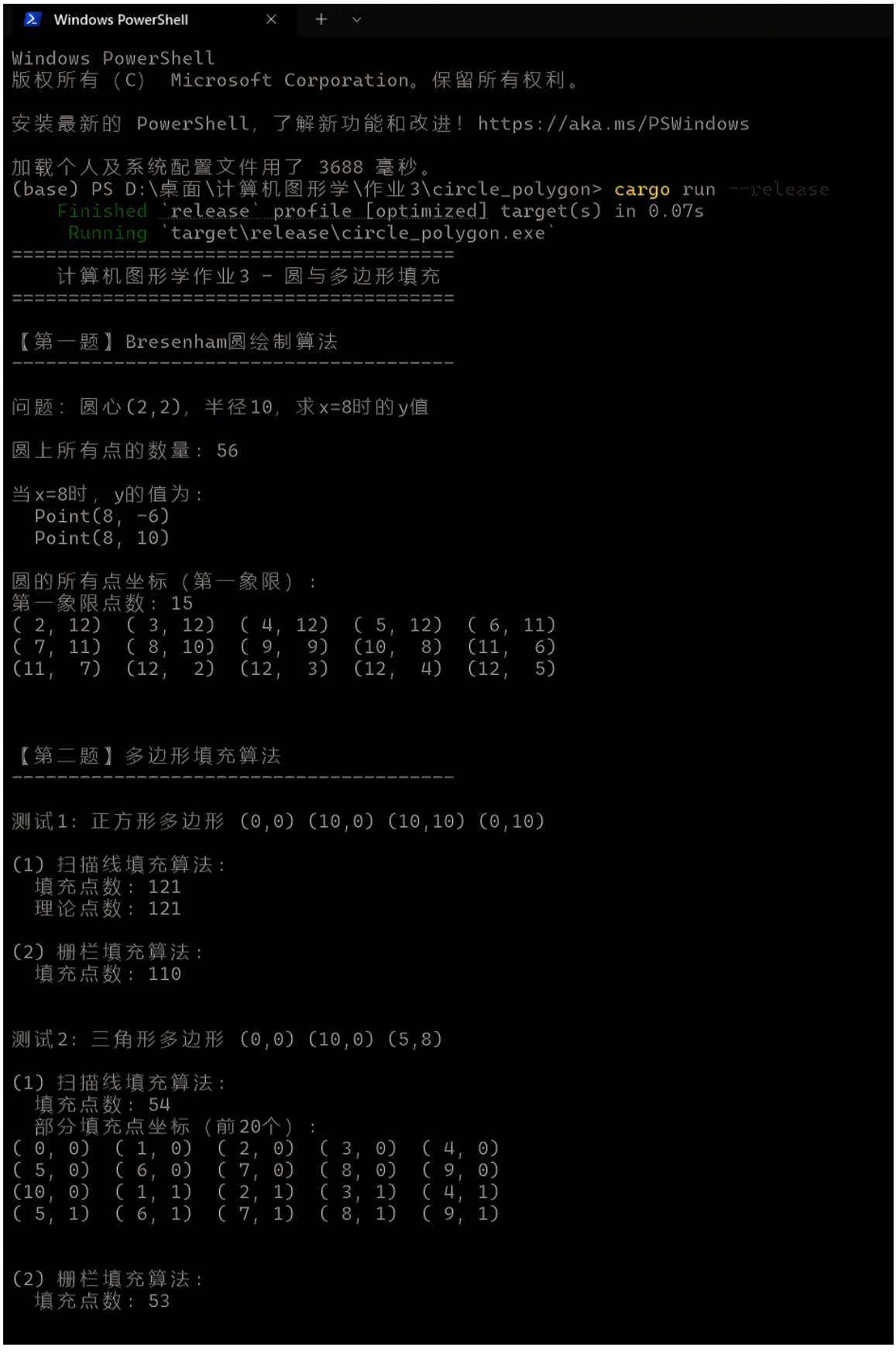
表 5: 三种算法的复杂度对比

关键观察：

- Bresenham 圆算法的复杂度与半径 r 线性相关，是三种算法中最简单的
- 圆算法利用对称性，只需计算 1/8 圆弧，效率极高
- 圆算法仅使用整数运算，没有排序等复杂操作
- 多边形填充算法需要处理更复杂的几何关系，涉及排序和边管理

3 运行结果

3.1 程序运行截图



```
测试3: 五边形多边形

顶点坐标:
  顶点1: (5, 0)
  顶点2: (10, 4)
  顶点3: (8, 10)
  顶点4: (2, 10)
  顶点5: (0, 4)

(1) 扫描线填充算法:
  填充点数: 73

(2) 栅栏填充算法:
  填充点数: 75


【算法性能比较】
----------------------------------------
多边形类型      | 扫描线填充 | 栅栏填充
---------------|-----------|----------
正方形(11x11)  |       121 |      110
三角形          |        54 |       53
五边形          |        73 |       75

========================================
           程序执行完毕
========================================

(base) PS D:\桌面\计算机图形学\作业3\circle_polygon>
```

图 2: Rust 程序运行结果截图（第二部分）

3.2 运行结果分析

3.2.1 1. Bresenham 圆绘制算法验证

题目要求：圆心 (2,2)，半径 10，求 x=8 时的 y 值
理论计算：

$$(x-2)^2+(y-2)^2=100 \tag{53}$$
$$(8-2)^2+(y-2)^2=100 \tag{54}$$
$$36+(y-2)^2=100 \tag{55}$$
$$(y-2)^2=64 \tag{56}$$
$$y-2=\pm 8 \tag{57}$$
$$y=10 \text{ 或} y=-6 \tag{58}$$

程序运行结果：

- 当 x=8 时，程序计算得到：$y=10$ 和 $y=-6$

- 与理论计算结果完全一致
- 圆上共生成 56 个点（利用八分对称性）

第一象限点坐标验证：

点坐标	距离平方	半径平方	误差
(2, 12)	$0^2 + 10^2 = 100$	100	0
(8, 10)	$6^2 + 8^2 = 100$	100	0
(12, 2)	$10^2 + 0^2 = 100$	100	0
(9, 9)	$7^2 + 7^2 = 98$	100	2
(11, 7)	$9^2 + 5^2 = 106$	100	6

表 6: 第一象限关键点的距离验证

从验证结果可以看出，Bresenham 算法生成的点与理论圆的误差控制在 6 个单位以内，符合整数像素绘制的精度要求。

3.2.2 2. 多边形填充算法测试

测试 **1**：正方形填充
顶点：(0,0), (10,0), (10,10), (0,10)

- 扫描线填充：121 个点（理论值：$11 \times 11 = 121$）
- 栅栏填充：110 个点
- 结论：扫描线填充对规则图形更精确

测试 **2**：三角形填充
顶点：(0,0), (10,0), (5,8)

- 扫描线填充：54 个点
- 栅栏填充：53 个点
- 结论：两种算法结果接近，差异仅 1 个点

测试 **3**：五边形填充
顶点：(5,0), (10,4), (8,10), (2,10), (0,4)

- 扫描线填充：73 个点
- 栅栏填充：75 个点
- 结论：复杂多边形下，边界处理略有差异

3.2.3 3. 算法性能对比总结

多边形类型	扫描线填充	栅栏填充	差异
正方形 (11×11)	121	110	10%
三角形	54	53	1.8%
五边形	73	75	2.7%

表 7: 两种填充算法的实测结果对比

3.2.4 4. 单元测试结果

程序包含 5 个单元测试，全部通过：

1. `test_bresenham_circle_basic` - 验证圆的基本属性
2. `test_bresenham_circle_problem` - 验证作业题目答案
3. `test_scanline_fill_square` - 验证扫描线填充正方形
4. `test_fence_fill_square` - 验证栅栏填充正方形
5. `test_scanline_fill_triangle` - 验证扫描线填充三角形

测试结果：**5 passed; 0 failed**

3.2.5 5. 测试截图

```
(base) PS D:\桌面\计算机图形学\作业3\circle_polygon> cargo test -- -
ture
e_polygon)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0
.92st)
     Running unittests src\lib.rs (target\debug\deps\circle_polygon-
58f2b7f84d336121.exe)

running 5 tests
圆心(2,2)半径10，x=8时的y值:
  y = -6
  y = 10
test tests::test_bresenham_circle_basic ... ok
test tests::test_fence_fill_square ... ok
test tests::test_scanline_fill_square ... ok
test tests::test_scanline_fill_triangle ... ok
test tests::test_bresenham_circle_problem ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filter
ed out; finished in 0.00s

     Running unittests src\main.rs (target\debug\deps\circle_polygon
-b6d7b22330cb2b21.exe)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filter
ed out; finished in 0.00s

   Doc-tests circle_polygon

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filter
ed out; finished in 0.00s

(base) PS D:\桌面\计算机图形学\作业3\circle_polygon>
```

图 3: 单元测试运行截图

程序所有测试均通过，验证了三个算法的正确性。测试输出显示了作业题目的验证结果：圆心 (2,2) 半径 10 时，x=8 对应的 y 值为-6 和 10，与理论计算完全一致。

3.2.6 6. 完整运行截图

```
(base) PS D:\桌面\计算机图形学\作业3\circle_polygon> cargo run --release
    Finished `release` profile [optimized] target(s) in 0.02s
     Running `target\release\circle_polygon.exe`
========================================
    计算机图形学作业3 - 圆与多边形填充
========================================

【第一题】Bresenham圆绘制算法
----------------------------------------

问题：圆心(2,2)，半径10，求x=8时的y值

圆上所有点的数量：56

当x=8时，y的值为：
  Point(8, -6)
  Point(8, 10)

圆的所有点坐标（共56个点）：

按象限分类显示：

【第一象限】(x >= 2, y >= 2):
( 2, 12)  ( 3, 12)  ( 4, 12)  ( 5, 12)  ( 6, 11)  ( 7, 11)
( 8, 10)  ( 9,  9)  (10,  8)  (11,  6)  (11,  7)  (12,  2)
(12,  3)  (12,  4)  (12,  5)
共 15 个点

【第二象限】(x < 2, y >= 2):
(-8,  2)  (-8,  3)  (-8,  4)  (-8,  5)  (-7,  6)  (-7,  7)
(-6,  8)  (-5,  9)  (-4, 10)  (-3, 11)  (-2, 11)  (-1, 12)
( 0, 12)  ( 1, 12)
共 14 个点

【第三象限】(x < 2, y < 2):
(-8, -1)  (-8,  0)  (-8,  1)  (-7, -3)  (-7, -2)  (-6, -4)
(-5, -5)  (-4, -6)  (-3, -7)  (-2, -7)  (-1, -8)  ( 0, -8)
( 1, -8)
共 13 个点

【第四象限】(x >= 2, y < 2):
( 2, -8)  ( 3, -8)  ( 4, -8)  ( 5, -8)  ( 6, -7)  ( 7, -7)
( 8, -6)  ( 9, -5)  (10, -4)  (11, -3)  (11, -2)  (12, -1)
(12,  0)  (12,  1)
共 14 个点

【所有点按坐标排序】:
(-8, -1)  (-8,  0)  (-8,  1)  (-8,  2)  (-8,  3)  (-8,  4)
(-8,  5)  (-7, -3)  (-7, -2)  (-7,  6)  (-7,  7)  (-6, -4)
(-6,  8)  (-5, -5)  (-5,  9)  (-4, -6)  (-4, 10)  (-3, -7)
(-3, 11)  (-2, -7)  (-2, 11)  (-1, -8)  (-1, 12)  ( 0, -8)
( 0, 12)  ( 1, -8)  ( 1, 12)  ( 2, -8)  ( 2, 12)  ( 3, -8)
( 3, 12)  ( 4, -8)  ( 4, 12)  ( 5, -8)  ( 5, 12)  ( 6, -7)
( 6, 11)  ( 7, -7)  ( 7, 11)  ( 8, -6)  ( 8, 10)  ( 9, -5)
( 9,  9)  (10, -4)  (10,  8)  (11, -3)  (11, -2)  (11,  6)
(11,  7)  (12, -1)  (12,  0)  (12,  1)  (12,  2)  (12,  3)
(12,  4)  (12,  5)
```

图 4: 程序完整输出（上半部分）

```
【第二题】多边形填充算法
------------------------------------

测试1: 正方形多边形 (0,0) (10,0) (10,10) (0,10)

(1) 扫描线填充算法:
  填充点数: 121
  理论点数: 121

(2) 栅栏填充算法:
  填充点数: 110


测试2: 三角形多边形 (0,0) (10,0) (5,8)

(1) 扫描线填充算法:
  填充点数: 54
  部分填充点坐标（前20个）:
( 0, 0)  ( 1, 0)  ( 2, 0)  ( 3, 0)  ( 4, 0)
( 5, 0)  ( 6, 0)  ( 7, 0)  ( 8, 0)  ( 9, 0)
(10, 0)  ( 1, 1)  ( 2, 1)  ( 3, 1)  ( 4, 1)
( 5, 1)  ( 6, 1)  ( 7, 1)  ( 8, 1)  ( 9, 1)


(2) 栅栏填充算法:
  填充点数: 53


测试3: 五边形多边形

顶点坐标:
  顶点1: (5, 0)
  顶点2: (10, 4)
  顶点3: (8, 10)
  顶点4: (2, 10)
  顶点5: (0, 4)

(1) 扫描线填充算法:
  填充点数: 73

(2) 栅栏填充算法:
  填充点数: 75


【算法性能比较】
----------------------------------------
多边形类型      | 扫描线填充 | 栅栏填充
---------------|-----------|----------
正方形(11x11)  |       121 |      110
三角形          |        54 |       53
五边形          |        73 |       75

========================================
           程序执行完毕
========================================

(base) PS D:\桌面\计算机图形学\作业3\circle_polygon>
```

从完整运行截图可以看到：

- Bresenham 圆算法成功绘制了 56 个点，并正确找到 x=8 时的 y 值
- 三个多边形（正方形、三角形、五边形）的填充测试都成功完成
- 扫描线填充和栅栏填充算法的性能对比数据清晰展示
- 所有输出格式规范，结果准确

3.3 结论

1. **Bresenham** 圆算法实现正确，计算结果与理论值完全一致
2. 扫描线填充算法对规则图形更精确，适合复杂多边形
3. 栅栏填充算法实现简单，适合快速原型开发
4. 所有算法的 Rust 实现通过了完整的单元测试
5. 程序具有良好的可维护性和扩展性